

BILAN ENTREES SORTIES Proc LPC1769 partie BASE						
NOM MODULE	NOM SIGNAL	ANA IN/ ANA OUT	PERIPHERIQUE	PIN FUNCTION	Px.y	Gestion par IT ou tâche / variables associées
encadré Schéma	Compatible Doc Elec	NUM IN / NUM OUT	NOM / NUMERO	id nom tab pinsel		
ECRAN DE SUPERVISION	BASE_TO_SUPER	NUM OUT	UART3	TXD3 (0b01) PINSELO	P0.0	Après l'acquisition du message "RXvS\r\n" ou "lettrePyy" de la poste, on ajoute "Pnn" en entête et on transmet le message ("PnnRXvS" ou "PnnlettrePyy") vers la supervision.
	SUPER_TO_BASE	NUM IN		RXD3 (0b01<<2) PINSELO	P0.1	La base reçoit des messages consigne vitesse de type "RXVyy\r\n" (X - ID robot, yy - vitesse, \r\n retour à la ligne qui indique le fin du message) et aussi des messages d'attribution de type "RXPss-Pdd\r\n" (X - ID robot, ss - numéro poste de départ, dd - numéro poste d'arrivé, \r\n retour à la ligne qui indique le fin du message). On va autoriser une interruption sur UART3, et la fonction UART3_IRQHandler() va gérer l'acquisition du message en le mettant dans une tableau buffer nommé rxBuffer[] jusqu'à atteindre le fin du message. On met à jour les tableaux : 1. si le message est de la forme "RXVyy\r\n" vitesse_voulue[X] = (rxBuffer[3]- '0')*10 + (rxBuffer[4]- '0'); (X - ID robot, 3 et 4 - position de la vitesse dans le message reçu). 2. si le message est de la forme "RXPss-Pdd\r\n" enlevement[X] = [rxBuffer[3];rxBuffer[4]]; et livraison[X] = [rxBuffer[7];rxBuffer[8]];
POSTES DE TRAVAIL	BASE_TO_POSTE	NUM OUT	UART0	TXD0 (0b01<<4) PINSELO	P0.2	La base envoie un message de type "@Tnn\r\n" (nn - numéro de robot, \r\n retour à la ligne qui indique le fin du message). On va créer une fonction UART0_envoye_char(char c); qui va envoyer un caractère si UART0->LSR[5] == 1, ça veut dire qu'on envoie si le registre de transmission est vide.
	POSTE_TO_BASE	NUM IN		RXD0 (0b01<<6) PINSELO	P0.3	La base reçoit des messages type "RXvS\r\n" (X - ID robot, v - vitesse, S - status, \r\n retour à la ligne qui indique le fin du message). On va autoriser une interruption sur UART0, et la fonction UART0_IRQHandler() va gérer l'acquisition du message en le mettant dans une tableau de type buffer nommé rxBuffer[] jusqu'à atteindre le fin du message. On met à jour les tableaux : 1. vitesse_actuelle[X] = rxBuffer[2] (X - ID robot, et 2 - position de la vitesse dans le message reçu); 2. on vérifie si le robot à change de status et on mis à jour les tableaux enlevement[] et livraison[]. Si status changé à enlevecolis, enlevement[X] = "00". Si status changé à livraison, livraison[X] = "00".
DIP SWITCH NOMBRE DE ROBOTS	NB_ROB[0]	NUM IN	GPIO0	GPIO Port 0.4 (0b00<<8) PINSELO	P0.4	On va autoriser les interruptions sur front montant et front descendant. Dans l'EINT3_Handler() on va mettre à jour la variable globale NB_ROBOTS à chaque changement d'état des DIP SWITCHs.
	NB_ROB[1]			GPIO Port 0.5 (0b00<<10) PINSELO	P0.5	
	NB_ROB[2]			GPIO Port 0.6 (0b00<<12) PINSELO	P0.6	
	NB_ROB[3]			GPIO Port 0.7 (0b00<<14) PINSELO	P0.7	
DIP SWITCH NOMBRE DE POSTES	NB_POSTE[0]	NUM IN	GPIO0	GPIO Port 0.8 (0b00<<16) PINSELO	P0.8	On va autoriser les interruptions sur front montant et front descendant. Dans l'EINT3_IRQHandler() on va mettre à jour la variable globale NB_POSTES à chaque changement d'état des DIP SWITCHs.
	NB_POSTE[1]			GPIO Port 0.9 (0b00<<18) PINSELO	P0.9	
	NB_POSTE[2]			GPIO Port 0.10 (0b00<<20) PINSELO	P0.10	
	NB_POSTE[3]			GPIO Port 0.11 (0b00<<22) PINSELO	P0.11	
FIL CONDUCTEUR	COURANT MODULE	NUM OUT	PWM1	PWM1.1 (0b10<<4) PINSEL3	P1.18	On va générer un signal PWM avec une fréquence de 50 kHz. On utilisera un Timer pour mesurer la durée pendant laquelle le PWM est actif (émission) ou inactif (pause). Pour envoyer l'entête ('E'), le PWM sera actif pendant 2,5 ms, puis désactivé pendant 0,5 ms. Pour envoyer un '0' logique, le PWM sera actif pendant 1,0 ms, puis désactivé pendant 0,5 ms. Pour envoyer un '1' logique, le PWM sera actif pendant 1,75 ms, puis désactivé pendant 0,5 ms.

BILAN ENTREES SORTIES Proc LPC1769 partie POSTE							
MODULE	NOM_signal	I/O	Peripherie	Fonction	Pin	Pinset	Gestion par IT ou tâche/variables associées
Clavier 16 touches	COLONNE_1	IN/Num	GPIO 0	GPIO port 0.0 (0b00)	P0.0	Pinset0	On connecte les 4 colonnes à des entrées numériques avec résistances de tirage interne (INPUT_PULLUP si disponible, ou externe). Si une touche est pressée, un contact entre une ligne (mise à 0) et une colonne se ferme, donc la colonne concernée passe à 0 → touche détectée. On utilise une tâche cyclique pour ce processus. (ou IT GPIO qu'on va essayer d'étudier)
	COLONNE_2	IN/Num	GPIO 0	GPIO port 0.1 (0b00<<2)	P0.1	Pinset0	
	COLONNE_3	IN/Num	GPIO 0	GPIO port 0.4 (0b00<<8)	P0.4	Pinset0	
	COLONNE_4	IN/Num	GPIO 0	GPIO port 0.5 (0b00<<10)	P0.5	Pinset0	
	LIGNE_1	OUT/Num	GPIO 0	GPIO port 0.6 (0b00<<12)	P0.6	Pinset0	On connecte les 4 lignes à des sorties numériques du microcontrôleur. Pour être activées une à une à l'état bas (0), ce qui permet d'identifier la rangée d'une touche pressée lors du balayage du clavier. On utilise une tâche cyclique pour ce processus.
	LIGNE_2	OUT/Num	GPIO 0	GPIO port 0.7 (0b00<<14)	P0.7	Pinset0	
	LIGNE_3	OUT/Num	GPIO 0	GPIO port 0.8 (0b00<<16)	P0.8	Pinset0	
	LIGNE_4	OUT/Num	GPIO 0	GPIO port 0.9 (0b00<<18)	P0.9	Pinset0	
Comm série avec base	BASE_TO_POSTE	IN/Num	UART0	TXD0 (0b01<<4)	P0.2	Pinset0	le poste ouvrier reçoit par UART en interruption RX le message @Tnn\r\n, le compare à son numéro (nn) lu sur ses entrées GPIO ; s'il est concerné, il prépare une réponse.
	POSTE_TO_BASE	OUT/Num	UART0	RXD0 (0b01<<6)	P0.3	Pinset0	le poste envoie en UART TX (dans la boucle principale ou IT TX) un message de 6 caractères (RXvD<, CPyy<, etc.) selon ce qu'il a vu (robot ou demande d'acheminement).
DTMF	DTMF_OUT	O/ANALOG	DAC	AOUT (0b10<<18)	P0.26	Pinset1	Émission d'un signal pour commander le robot. Tâche.
IR	sgn_IR	IN/Num	CAP 3	CAP3.0 port 0.23 (0b11 << 14)	P0.23	Pinset1	A chaque passage d'un robot, une interruption externe est déclenchée (front de début de trame IR), puis un timer capture ou lecture périodique est utilisée pour décoder les bits de la trame (ID robot, vitesse, status).
Oscilloscope	start_seq_seen	OUT/Num	GPIO 1	GPIO port 1.28 (0b00<<24)	P1.28	Pinset3	Visualisation du signal sur l'oscilloscope pour vérification. Top pour synchroniser l'oscilloscope
LED Vitesse	LED_v1	OUT/Num	GPIO 0	GPIO port 0.24 (0b00<<16)	P0.24	Pinset1	Affichage du vitesse du robot a travers les LED en binaire. Tâche cyclique.
	LED_v2	OUT/Num	GPIO 0	GPIO port 0.25 (0b00<<18)	P0.25	Pinset1	
LED Status	LED_v3	OUT/Num	GPIO 0	GPIO port 0.17 (0b00<<2)	P0.17	Pinset1	Affichage status de robot sur les LED en binaire. Tâche cyclique.
	LED_v4	OUT/Num	GPIO 0	GPIO port 0.18 (0b00<<4)	P0.18	Pinset1	
	LED_s1	OUT/Num	GPIO 1	GPIO port 2.1 (0b00<<2)	P2.1	Pinset4	
	LED_s2	OUT/Num	GPIO 1	GPIO port 2.2 (0b00<<4)	P2.2	Pinset4	
LED Num Robot	LED_s3	OUT/Num	GPIO 1	GPIO port 2.3 (0b00<<6)	P2.3	Pinset4	Affichage du numéro du robot a travers les LED en binaire. Tâche cyclique.
	LED_s4	OUT/Num	GPIO 1	GPIO port 2.4 (0b00<<8)	P2.4	Pinset4	
	LED_r1	OUT/Num	GPIO 1	GPIO port 1.20 (0b00<<4)	P1.20	Pinset3	
	LED_r2	OUT/Num	GPIO 1	GPIO port 1.18 (0b00<<6)	P1.18	Pinset3	
LED Num Robot	LED_r3	OUT/Num	GPIO 1	GPIO port 1.19 (0b00<<8)	P1.19	Pinset3	Affichage du numéro du robot a travers les LED en binaire. Tâche cyclique.
	LED_r4	OUT/Num	GPIO 1	GPIO port 1.22 (0b00<<12)	P1.22	Pinset3	
LED Affichage du trame	LED_trame	OUT/Num	GPIO 2	GPIO port 1.25 (0b00<<18)	P1.25	Pinset3	Allumer seulement si la trame est validée, une fois la trame reçue et validée, le poste prépare une réponse série comme RXvD<, BXvC<, etc. à transmettre à la base lors de la prochaine interrogation @Tnn.
ID_POSTE	POSTE[0]	IN/Num	GPIO 1	GPIO PORT 2.5 (0b00<<10)	P2.5	Pinset4	Configuration de l'ID du poste à travers des switches.
	POSTE[1]	IN/Num	GPIO 1	GPIO PORT 2.6 (0b00<<12)	P2.6	Pinset4	
	POSTE[2]	IN/Num	GPIO 1	GPIO PORT 2.7 (0b00<<14)	P2.7	Pinset4	
	POSTE[3]	IN/Num	GPIO 1	GPIO PORT 2.8 (0b00<<16)	P2.8	Pinset4	

BILAN ENTREES SORTIES Proc LPC1769 partie ROBOT									
MODULE	NOM_signal	I/O	Peripherie	Fonction	Pin	Pinset	Notes		
Boutons	Charger	I/NUM	GPIO 0	GPIO port 0.9 (0b00<<18)	P 0.9	Pinset0	Reprise de trajet ou changement de statut - Déclanche interruption sur front montant/décroissant. Gérer dans EINT3_IRQHandler()		
	Décharger		GPIO 2	GPIO port 2.1 (0b00<<2)	P 2.1	Pinset4			
Coupe circuit	Coupe_C	I/ANALOG	GPIO 0	GPIO port 0.4 (0b00<<8)	P 0.4	Pinset0	Indique un arrêt d'urgence (coupe circuit)		
Mémoire	Reg_DATA	I/NUM	UART3	TXD3 (0b01<<4)	P 0.2	Pinset0	registre contenant les messages sonores		
	Reg_ADR	O/NUM		RXD3 (0b01<<6)	P 0.3	Pinset0	registre contenant les adresses des messages sonores		
Micro switch de debug	Debug Télémètre ultrason	I/NUM	GPIO 0	GPIO port 0.5 (0b00<<10)	P 0.5	Pinset0	choisir un mode de debug pour le télémètre afin de mesurer la distance avec un obstacle en faisant une combinaison de 2 switchs		
	Debug commande des roues			GPIO port 0.6 (0b00<<12)	P 0.6		choisir un mode de debug pour le télémètre afin de commander les roues du robot en faisant une combinaison de 2 switchs		
	Debug Capteurs inductifs			GPIO port 0.7 (0b00<<14)	P 0.7	Pinset1			
				GPIO port 0.8 (0b00<<16)	P 0.8	choisir un mode de debug pour le détecteur de position en faisant une combinaison de 3 switchs			
				GPIO port 0.16 (0b00<<0)	P 0.16				
				GPIO port 0.19 (0b00<<6)	P 0.19				
	Debug série OUT	O/NUM	UART0	TXD0 (0b01<<4)	P 0.0	Pinset0	Envoie message de debugs - Programmer la communication UART pour le débogage		
	Debug série IN	I/NUM		RXD0 (0b01<<6)	P 0.1	Pinset0	Programmer la communication UART pour le débogage		
Emetteur IR	SGN_IR	O/ANALOG	PWM 1	PWM1.1 (0b10<<0)	P 2.0	Pinset4	émission (num robot , vitesse et status), Ce signal est généré sur 18 bit (2 bit pour l'entête, 12 bit d'information: les 4 premiers : numeros de robot, 4 bits: sa vitesse, 4bits: son statut et 4 bit de controle), la transmission des 0 et 1 logique ce fait avec des motifs sur 3 "tau" . Cette durée "tau" est gérée par un Timer, on programme les MR et les timers		
	Top_synchro_IR	O/NUM	GPIO0	GPIO PORT 0.21 (0b00<<10)	P 0.21	Pinset1	TOP Synchronisation (trame_valide)		
Télémètre ultrason	Telemetre_PWM	O/NUM	GPIO 1	GPIO Port 1.20 (0b00<<8)	P 1.20	Pinset 3	PWM réalisé avec timer qui rentre dans l'oscillateur pour le télémètre US		
	Buzzer_PWM		GPIO 0	GPIO Port 0.10 (0b00<<20)	P 0.10	Pinset 0	PWM réalise avec systick qui rentre dans l'oscillateur pour le Buzzer Alarme		
	Telemetre_Recep	I/NUM	CAP0	CAP 0.1 (0b00<<22)	P 1.27	Pinset 3	Capture utilisé pour mesuré le déphasage entre le Telemetre_PWM et le signal reçu dans la		
LED RGB	SGN_RGB	O/NUM	GPIO 0	GPIO Port 0.22 (0b00<<12)	P 0.22	Pinset1	indique le statut		
			GPIO 3	GPIO Port 3.25 (0b00<<18)	P 3.25	Pinset7			
SERVOMOTEUR	PWM_SERVO	O/NUM	GPIO 0	GPIO Port 0.23 (0b00<<14)	P 0.23	Pinset1	indique la lettre du livreur (A/B/C/D). PWM conçu avec un systick (interval = 1ms)		
Haut Parleur	SGN_hp	O/ANALOG	DAC	AOUT (0b10<<20)	P 0.26	Pinset1	prevention de livraison en message sonore pour avertir un ouvrier qu'il doit intervenir et charger ou décharger le robot		
Récepteur DTMF(HT9170D)	DTMF_IN1	IN/Num	GPIO 2	GPIO port 2.2 (0b00<<4)	P 2.2	Pinset4	Il utilise des techniques de comptage numérique pour détecter et décoder toutes les 16 paires de tonalités DTMF en une sortie codée sur 4 bits. C'est un décodage par IT via HT9170D		
	DTMF_IN2			GPIO port 2.3 (0b00<<6)	P 2.3				
	DTMF_IN3			GPIO port 2.4 (0b00<<8)	P 2.4				
	DTMF_IN4			GPIO port 2.5 (0b00<<10)	P 2.5				
	DTMF_OUT1	O/Num		GPIO port 2.6 (0b00<<12)	P 2.6		Sorties sur des LEDS pour indiquer le poste reconnu		
	DTMF_OUT2			GPIO port 2.7 (0b00<<14)	P 2.7				
	DTMF_OUT3			GPIO port 2.8 (0b00<<16)	P 2.8				
	DTMF_OUT4			GPIO Port 2.12 (0b00<<24)	P 2.12				
Capteurs inductifs	DTMF_IN_Value_Ready	IN/Num		GPIO Port 2.11 (0b00<<22)	P 2.11		Front montant quand il y a une nouvelle valeur à lire		
	V_bob_avant	I/ANALOG	AD 0	AD0.2 (0b01<<18)	P 0.25	Pinset1	récupère la tension aux bornes de la bobine horizontale avant pour mesurer la distance		
	V_bob_arriere	I/ANALOG	AD 0	AD0.5 (0b11<<30)	P 1.31	Pinset3	récupère la tension aux bornes de la bobine arrière avant pour mesurer la distance		
	Top_synchro_Courant	O/NUM	GPIO 0	GPIO port 0.29 (0b00<<26)	P 0.29	Pinset1	TOP Synchronisation entête reconnu		
	XOR1_VERT_HOR1	I/NUM	CAP1	Cap 1.0 (0b10<<4)	P 1.18	Pinset3	Capture utilisé pour mesuré l'angle entre le robot et le fil		
	XOR2_VERT_HOR2	I/NUM	CAP1	Cap 1.1 (0b10<<6)	P 1.19	Pinset3			
Protocole SPI liaison FPGA micro controleur	CAP_MOD_COURANT	I/NUM	CAP0	CAP0.0	P 1.26	Pinset3	Entrée Signal dans le fil démodulé		
	MISO	IN/NUM	SPI 0	MISO0(0b10<<2)	P 0.17	Pinset1	recoie la distance parcourue depuis FPGA		
	MOSI	OUT/NUM	SPI 0	MOSI0 (0b10<<4)	P 0.18		envoie requête pour avoir la distance parcourue depuis FPGA		
	CLK_MC	OUT/NUM	SPI 0	SCK0 (0b10<<30)	P 0.15	Pinset0	Horloge du MC pour cadencer la transmission SPI		
Commande des roues	CS	OUT/NUM	GPIO 0	GPIO port 0.24 (0b00<<16)	P 0.24	Pinset1	le signal chipselect pour determiner avec quel peripherique on communique		
	vitesse_wg	OUT/NUM	PWM 1	PWM1.5 (0b10<< 16)	P 1.24	Pinset3	définir les rapports cycliques pour les vitesses angulaires en tenant compte de l'écart entre les 2 roues (e), et le rayon des roues (r)		
Micro switch	vitesse_wd	OUT/NUM	PWM 1	PWM1.3 (0b10<< 10)	P 1.21				
	dip_switch[1]	I/NUM	GPIO 1	GPIO Port 1.30 (0b00<<28)	P 1.30	Pinset3	Switch de configuration de numero de robots. On autorise les interruptions sur front montant et front descendant. Dans l'EINT3_IRQHandler() on va mettre à jour la variable globale dip_switch à chaque changement d'état des switchs.		
	dip_switch[2]		GPIO 2	GPIO Port 2.1 (0b00<<2)	P 2.1	Pinset4			
	dip_switch[3]			GPIO Port 2.10 (0b00<<20)	P 2.10				
	dip_switch[4]			GPIO Port 2.13 (0b00<<26)	P 2.13				